When the Checker Is Noisier Than the Model: Measuring Number Faithfulness in LLM-Written Business Notes

Harshitha Venkatesh

Project page: https://harshitha-venkatesh-2204.github.io/NumGate/
Code and data: https://github.com/harshitha-venkatesh-2204/NumGate

A PREPRINT · October 20, 2026

Abstract

Executives increasingly read weekly notes that large language models (LLMs) draft from sales tables, where one wrong number can mislead a decision. We present NumGate, an open harness that checks every number in such a note against its source table with a two-level rule-based gate (is the number in the table or derivable from it, and is it attached to the right entity, metric and period), and compares four generation strategies on 60 weekly tables built from the public UCI Online Retail II data. In a pre-registered study of 1,440 notes from three Claude models (Opus 5, Sonnet 5 and Haiku 4.5), every number the gate flagged on the 54 tables not used in the pilot, and a random sample of unflagged numbers, was reviewed blind by AI agents, not people. Confirmed errors are rare: 0.22% of checked numbers for Opus 5, 0.05% for Sonnet 5 and 0.47% for Haiku 4.5. These are lower bounds, because only flagged numbers can be confirmed, and the sample of unflagged numbers cannot exclude true rates of up to about 2.44% to 2.92%. The main result concerns measurement. Only 30.6% of the gate's flags were real errors, and its precision depended on the writer, from 43.1% for Haiku 4.5 to 7.6% for Sonnet 5, where one phrasing that the gate misreads, a gap held-out testing had already found, caused most false alarms. Raw gate rates cannot tell Sonnet 5 from Opus 5. Compute-then-write raised Opus 5's error rate through rounding and mislabelled directions in its own facts script. Write-then-verify and a facts template reduced Haiku 4.5's errors under a conditional test and a sign test, although the pre-registered test no longer separates them once rounding slips are set aside; for Opus 5 their reductions rest on six rounding slips and are not robust. Faithfulness rates from automatic checkers should be reported with the checker's precision measured on the same text.

Keywords LLM evaluation · number faithfulness · data-to-text generation · hallucination detection · measurement error

1 Introduction

Weekly business reviews summarise tables: revenue by country this week (TW) versus last week (LW) and the same week last year (LY), week over week (WoW) and year over year (YoY) changes, shares and ranks. LLMs are now used to draft these notes. The failure that matters most in this setting is not fluency or tone but a wrong number: a revenue figure attributed to the wrong country, a decline reported as growth, or a total that no row of the table supports.

Measuring this failure is harder than it looks. A note can cite a number that is not a cell but is correctly derived (a share, a total, a percent change), cite a correct number with the wrong label (last year's figure presented as last week's), or state the right number for the wrong entity. Scoring notes by hand does not

scale across models, strategies and table sizes, and an automatic checker has its own error rate.

We ask four questions:

RQ1 How often are the numbers in LLM-written notes wrong, either unsupported by the table or attached to the wrong entity, metric or period?

RQ2 How accurate is an automatic checker of those numbers on real notes, and does its accuracy depend on the writer?

RQ3 Which generation strategy reduces errors most: direct writing, compute-then-write with code execution, write-then-verify, or a deterministic facts template?

RQ4 How do error rates change with table size and model?

This paper describes the measurement instrument, a pilot study with one model on six tables that shaped the instrument, and a pre-registered full study with three models on all 60 tables that answers RQ1 to RQ4. Our contributions are:

- **NumGate**, a reproducible harness: a table builder over public retail data, four generation strategies behind one model interface with on-disk caching and a budget stop, a two-level number gate, analysis with bootstrap confidence intervals, and an interactive results dashboard.
- **A rule-based number gate** whose rules fit in two pages (Section 5) and whose verdicts carry evidence (the table cell or the operation that supports each number).
- **Pilot results** for Claude Opus 5 across the four strategies, at the number, claim and note level, with every flag checked against the tables.
- **A pre-registered full study** (Section 9): 1,440 notes from Claude Opus 5, Sonnet 5 and Haiku 4.5, in which every flagged number and a random sample of unflagged numbers were reviewed blind by AI agents, so that the reported error rates are confirmed errors rather than raw checker output, with deviations from the plan and exploratory analyses labelled.
- **A measurement lesson**, first seen in the pilot and confirmed at scale: a first version of the gate overstated the direct-writing claim error rate about 16 times, and the final gate's precision on real notes ranges from 7.6% to 43.1% depending on the writer, so raw checker rates cannot be compared across writers. We recommend that papers reporting LLM faithfulness rates also report the precision of their checker on the same text.

2 Related work

Faithfulness in data-to-text generation. Table-to-text generation has long been evaluated for faithfulness to the source. Wiseman et al. [31] introduced extractive evaluation of generated game summaries, including a relation-generation measure that extracts (entity, value, type) records from text and checks them against the source table, which is the closest precedent to our Level 2. ToTTo [24] and PARENT [8] address faithful table-to-text generation and its evaluation. TabFact [5] frames the verification of statements against tables as a classification task. Our setting differs in three ways: the text is an executive note with many derived quantities, the checker is fully deterministic apart from claim extraction, and we evaluate generation strategies rather than models alone.

Hallucination detection. Surveys document how often generated text states unsupported facts [16]. FActScore [22] decomposes long-form text into atomic facts and checks each, and SelfCheckGPT [21] detects hallucination from sampling consistency. NumGate follows the decompose-and-check pattern, but its source of truth is a small table, so support can be decided exactly with a rounding rule.

Numbers in generated text. Quantities are a known weak spot of generated text. HERMAN [32] verifies the quantities in a beam of candidate summaries and promotes the candidates whose quantities the source supports, and Puduppully et al. [25] improve generation from RotoWire box scores through content selection and planning. Proof-Carrying Numbers [27] is the idea closest to our gate: the model emits numbers bound to structured claims, and a verifier checks each against a declared matching policy, rounding included, before it is shown as verified. NumGate instead checks free-text notes after they are written, and asks how often such a check is right. For numerical claims checked against evidence, QuanTemp [30] is a benchmark of real-world numerical claims from fact-checking sources, Aarnes and Setty [1] make LLM claim verifiers robust to perturbed numbers, TabVer [2] verifies claims against tables with natural-logic proofs that include arithmetic over cells, and FinDVer [33] benchmarks claim verification over long financial documents that mix text and tables. Closest to our business setting, Goldsack et al. [14] generate analytical reports from earnings calls with LLMs and compare them with reports written by human analysts.

Evaluating the evaluators. Meta-evaluation benchmarks for summarisation factuality, among them FRANK [23], TRUE [15], SummaC [19] and AggreFact [28], measure automatic factuality detectors against human labels; the last finds that detectors which do well on older summarisers do worse on the outputs of recent ones. For data-to-text, Thomson and Reiter [29] propose a human gold-standard protocol for accuracy errors, and Kasner and Dušek [17] annotate semantic accuracy errors in LLM outputs with both human annotators and an LLM-based metric. Godbole and Jia [13] show that leading factuality metrics disagree with one another, misestimate system-level performance and should be validated in the target domain before use; our results make the same point for a rule-based numeric checker. LLMs are now widely used as judges [34], which makes the reliability of their verdicts a question in its own right: Krumdick et al. [18] find that LLM judges without a correct reference agree with expert graders mainly on questions they can answer correctly themselves. In this paper both the reviewers and the writers are LLMs. Our central lesson, that a checker's precision falls when true errors are rare, is the positive predictive value of a screening test. Epidemiology corrects prevalence estimates for an imperfect test [26], and prediction-powered inference [3] combines a small set of gold labels with many model predictions to give valid intervals. In text generation, Deriu et al. [7] model the errors of automated metrics so that a small set of human ratings corrects system rankings, and Gligorić et al. [12] use LLM annotations and their confidence to decide which items need human labels while keeping conclusions valid. We apply none of these corrections, and Section 10 discusses what they would add.

Computing before writing, and verifying after. Program-aided language models [11] and Program of Thoughts [6] have the model write code whose execution produces the numbers, which motivates our compute-then-write strategy. Self-Refine [20] and Chain-of-Verification [9] have the model critique or verify its own draft; our write-then-verify strategy instead feeds back the flags of an external deterministic checker.

3 Task and data

Source. We use UCI Online Retail II [4]: invoice lines of a UK online gift retailer from December 2009 to December 2011, with invoice, stock code, description, quantity, date, unit price, customer and country. The data come as two yearly sheets that overlap in early December 2010; keeping each invoice once leaves 1,044,848 lines. We drop cancelled invoices, rows with non-positive quantity or price, rows without a country and non-product stock codes (postage, bank charges and similar), leaving 1,014,193 rows. Revenue is quantity times unit price, in GBP. Dropping a cancellation does not remove the sale it reversed, so revenue is gross of returns, and in one table the top product is a large order that was later reversed. Products are identified by description, so a product whose description changed appears as two rows.

Report tables. We aggregate to ISO weeks. Each table lists the top N entities of one week by revenue, with columns Entity, Revenue_TW, Revenue_LW, Revenue_LY, Units_TW, Units_LW, Orders_TW, Orders_LW, WoW_Pct, YoY_Pct, Share_Pct and Rank. All derived columns are computed from the rounded stored columns, so every derived cell recomputes exactly from the CSV. Tables come in three sizes: small (5 to 10 rows), medium (20 to 40) and large (80 to 150). Countries fill only small tables (a typical week has about 13 trading countries), so medium and large tables list products. With a fixed seed we build 60 tables: 20 small (10 country, 10 product), 20 medium and 20 large. Totals and shares cover only the rows shown, which matters for notes that speak of "total revenue". Last-year columns are empty when the same week of the previous year falls outside the data, as it does for 29 of the 60 tables, and a last-week or last-year value of zero can also mean that a product's description changed.

4 Generation strategies

All strategies produce the same note format, enforced by the prompt: 6 to 8 bullet lines, one insight per line, each naming an entity and containing at least one number, with periods labelled explicitly (TW, LW, LY, WoW, YoY), facts only and no recommendations. All prompts are released with the harness.

S1 Direct. The table is given as a markdown table; the model writes the note in one call.

S2 Compute-then-write. The model writes a short pandas script that computes up to 25 facts (entity, metric, period, value, formula). The script runs in a subprocess with a 30 second timeout; on failure it is retried once with the error message. A second call writes the note from the facts alone, without the table.

S3 Write-then-verify. The S1 draft is scored by the Level 1 gate; unsupported numbers are sent back with the table for correction or removal, for at most two rounds.

S4 Facts-template. A deterministic pandas facts packet (totals, top three entities, largest WoW and YoY movers, most orders) is narrated with every number copied verbatim. S4 is not a free-generation strategy: it shows what narrating precomputed facts achieves.

5 The number gate

The gate is deterministic except for claim extraction. Full rules with worked examples are in the released method document (`docs/method.md`); we summarise them here.

Level 1: number support. A single regular expression extracts every numeric mention with its value, cited precision, unit (currency, percent, count, plain), sign and hedge. Signs come from explicit signs (including an en dash used as a minus) or from direction words (fell, rose, down) before or after the number, but a level word (to, from, at) between a direction word and the number makes it a level: in "fell

to £1,632.90 TW" the amount is unsigned. Numbers that are not measurements are skipped with a recorded reason: digits inside entity names, week labels, years and counting phrases ("top 3", "2 of the 5 countries"). A cited value c matches a true value t when t rounds half-up to it at the cited precision, $-h \leq |t|/s - |c| < h$ with $h = 0.5 \cdot 10^{-d}$ for scale s and d decimals, and signed values agree in sign; values written with k, M or B also match within 0.5 percent, hedged values ("about") within 5 percent. Each number is compared, in order, with (1) cells of the entities named in its line, (2) values derived from those entities and from the groups and product families the line mentions (changes, shares, share-point changes, per-order values, totals of the rest of the table, family sums), (3) table-level aggregates when the line names no entity or uses an aggregate word, and (4) any other cell. The first match labels the number Supported_cell or Supported_derived with its evidence; otherwise it is Unsupported. Anchoring derived values on the entities named in the line keeps the candidate set small: in a 150-row table, unrestricted pairwise operations would match almost any number by chance.

Level 2: claim binding. A cheap model (Claude Haiku 4.5 in all our runs; it therefore also reads its own notes) extracts one claim per number, (entity, metric, period, value, direction, qualifier), with a fixed JSON schema and prompt. The extractor only transcribes; values are re-parsed by the Level 1 parser. Labels written in the note override the extractor: a period label adjacent to a value ("£2,729.22 LW") sets its period, change wording ("a decline of £51.1k") marks it as a change, and a unit noun ("8 orders") sets its metric; values are located only where they stand alone, so 33 is not found inside £680.33. Each claim is resolved to one or more rows (exact name, aliases and demonyms, word-subset match, fuzzy match with ties kept) and to a column, and labelled Correct; Wrong_value if only the sign differs or nothing matches; Wrong_metric if the value belongs to another column of the same row (for example the YoY figure cited as WoW); Wrong_entity if it belongs to the same column of another row that the line names, or of exactly one other row; and Unverifiable for names that fit several rows none of which matches and for metrics the table cannot answer. Group figures ("the three parasol lines combined", "the top five"), including subsets the extractor mislabels as the whole-table total, are checked against the same group, product-family, top-k and table totals that Level 1 uses for the line, and are labelled Correct or Wrong_value.

Metrics. The *unsupported rate* is Unsupported over checked numbers. The *binding error rate* is (Wrong_entity + Wrong_metric) over verifiable claims. The *claim error rate* adds Wrong_value. Rates are pooled over the notes of a group, with 95 percent percentile bootstrap intervals from 1,000 resamples of notes [10]; strategy differences use a paired bootstrap over conditions matched on table, model and repeat. The full study (Section 9) resamples tables instead of notes, because notes written on the same table are not independent.

6 Pilot study

Setup. The pilot runs Claude Opus 5 on the smoke set: two tables of each size (60-table pool, seeded selection), four strategies and two repeats, 48 notes in all. Opus 5 does not accept a temperature parameter and thinks adaptively by default, so repeats sample independently. Every response is cached, and the repeat index is part of the cache key. The pilot's note generation cost $2.54 ($0.053 per note on average).

Scale of the pilot. The 48 notes contain 1,730 numeric mentions: 1,625 were checked and 105 skipped (76 inside entity names, 21 counting phrases, 8 week labels). Of the checked numbers, 1,367 matched a cell and 255 a derived value. The extractor returned 1,613 claims, of which 1,613 were verifiable, including 69 group figures checked against group totals.

Table 1. Pilot results for Claude Opus 5 (48 notes; gate v3). Rates pool numbers or claims over notes; brackets give 95 percent bootstrap intervals over notes. With 12 notes per strategy the intervals are wide, and a cell with no event gets a zero-width interval that does not bound its rate.

Strategy	Numbers	Unsupported %	Claims	Wrong entity or metric %	Any claim error %	Notes with an error	Cost per note
S1 Direct	514	0.6 [0.0, 2.0]	508	0.0 [0.0, 0.0]	0.6 [0.0, 2.2]	1 / 12	$0.049
S2 Compute-then-write	301	0.0 [0.0, 0.0]	301	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.095
S3 Write-then-verify	514	0.0 [0.0, 0.0]	508	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.053
S4 Facts-template	296	0.0 [0.0, 0.0]	296	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.015

Results. Table 1 gives the pilot rates. Opus 5 cited very few unsupported numbers: 3 of 514 under direct writing (0.6%), all in one sentence that reported a wrong total for a product family, and none under the other strategies. At the note level, which is what a reader of the note experiences, 1 of 12 direct-writing notes contained a wrong sentence and no note under the other strategies did. No claim under any strategy attached a number to the wrong entity, metric or period; the only claim errors are the same three group figures. Section 8 checks these verdicts against the tables. Write-then-verify revised 1 of 12 drafts: the draft that contained the pilot's only real error, a total for six parasol and umbrella lines. The revision did not correct that total; it replaced the claim with a narrower one, a total for the three EDWARDIAN PARASOL colourways only (£9,139.25 TW versus £9,454.25 LW, a 3.3% decline), which is correct. Removing or narrowing a flagged claim, rather than fixing it, is a behaviour a verify loop can reward. The facts template produced the shortest and cheapest notes. Table 2 breaks the pilot down by table size: all errors fall in one medium table, and with two tables per size no size effect can be read from it. 4 of 48 notes broke the 6 to 8 bullet rule, all on one small country table: both of its direct-writing generations ended in a refusal partway through (the API returned the stop reason "refusal"), and write-then-verify reused both truncated drafts. These rates are far lower than the ones our first gate reported, which is the subject of the next section.

Table 2. Pilot results by table size (all four strategies pooled; two tables per size).

Table size	Tables	Notes	Numbers	Unsupported (%)	Claims	Claim errors	Notes with an error
small	2	16	440	0 (0.0)	438	0	0 / 16
medium	2	16	576	3 (0.5)	572	3	1 / 16
large	2	16	609	0 (0.0)	603	0	0 / 16

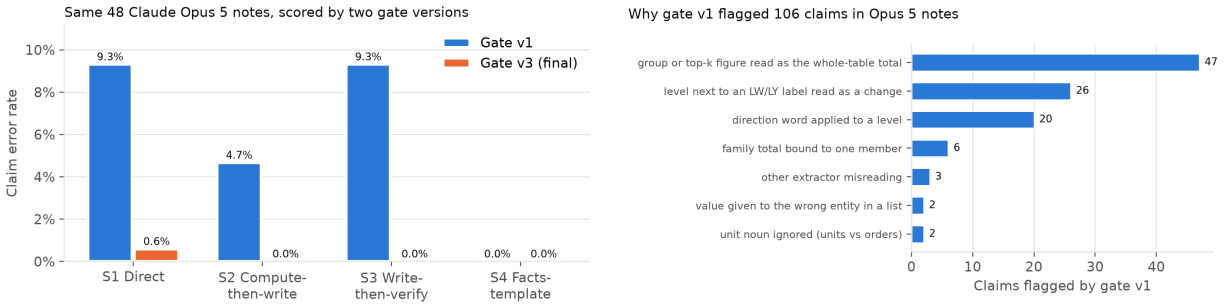


Figure 1. Left: claim error rate on the same 48 Opus 5 notes under gate v1 and the final gate (v3). Right: why gate v1 flagged its 106 claims. Almost all v1 flags were false alarms caused by ordinary phrasing.

7 How much of the measured error is the checker's?

The first version of the gate (v1) was developed on a mock model that writes template notes with planted errors, where it caught every planted error. On the real Opus 5 notes it reported claim error rates of 9.3% (S1), 4.7% (S2) and 9.3% (S3), and an S1 unsupported rate of 2.1%. Reading the flagged sentences showed that nearly all were correct. Figure 1 categorises the 106 flagged claims:

- **Group or top-k figures read as the whole-table total** (47). "The top five TW products combined for 16.9% share" was extracted with entity TOTAL and compared with the total's 100 percent share.
- **A level next to an LW or LY label read as a change** (26). In "£3,205.71 TW versus £2,729.22 LW (WoW +17.5%)" the extractor gave the LW amount the period WoW.
- **A direction word applied to a level** (20). In "fell to £1,632.90 TW" the level was signed negative and failed to match.
- **Family totals bound to one member, unit nouns ignored, list order** (13 together): "the three EDWARDIAN PARASOL colourways combined" matched the BLACK product; "140 units and 8 orders" gave 8 the metric units; a "respectively" list was assigned out of order.

The fixes are small and general: directions sign only changes; a label written next to a value overrides the extractor; the extractor marks group figures as GROUP and Level 1 knows family and group totals; ambiguous short names are checked against every row they fit; and a same-row mix-up is tested before a different-row one. Scored with the final gate (v3; gate v2 was not kept in a form that reproduces its own pilot scores), S1's claim error rate on the same notes is 0.6%, and Level 1 flags fall from 22 to 3.

The lesson generalises beyond this harness. When a model's true error rate is around one percent, a checker with a false alarm rate of a few percent per claim produces a measured error rate several times the true one, and differences between strategies can be artefacts of which phrasings each strategy tends to use. A checker validated only on synthetic errors (our mock) can look perfect and still fail on real text.

8 Validating the gate

The gate went through three versions. Gate v1 was built on the mock model. Gate v2 was revised after reading v1's flags on the pilot, and was then measured on a first set of fresh held-out notes. Gate v3 was revised using that held-out set's diagnoses and the pilot adjudication, frozen (its source file's checksum is recorded), and measured on a second held-out set written on tables not used by the stress suites or the pilot. Each held-out set therefore measures a gate that was not revised on it; every other row of Table 3 measures a gate on text its rules were revised on. The full study then measures the frozen gate v3 on 1,440 real notes by blind review (Section 9).

Table 3. Accuracy of the gate on its test sets. False alarms are correct numbers (L1) or claims (L2) flagged as errors; caught errors are deliberately wrong (or, for the pilot, adjudicated wrong) numbers and claims that were flagged. Only the held-out rows measure a gate on text its rules were not revised on.

Test set	L1 false alarms (%)	L1 errors caught (%)	L2 false alarms (%)	L2 errors detected (%)
Regression suites, gate v3 (rules tuned on them)	2 / 291 (0.7)	61 / 88 (69)	0 / 149 (0.0)	49 / 49 (100)
Held-out set 1, gate v2	22 / 353 (6.2)	79 / 111 (71)	8 / 202 (4.0)	75 / 77 (97)
Held-out set 2, gate v3	7 / 435 (1.6)	90 / 146 (62)	6 / 270 (2.2)	99 / 104 (95)
Opus pilot, gate v3, adjudicated (rules revised on it)	0 / 1,622 (0.0)	3 / 3 (100)	0 / 1,610 (0.0)	3 / 3 (100)

Regression stress suites. Two suites written in varied LLM styles, with ground truth computed from the tables, test Level 1 on 379 numbers and Level 2 on 203 claims. Gate v3 raises 2 false alarms on 291 correct numbers and catches 61 of 88 wrong ones; most of the misses are real cells attached to the wrong row or period, which Level 1 cannot see by design and Level 2 catches. At Level 2 it raises 0 false alarms and detects 49 of 49 wrong claims. Because the fixes were developed while looking at these suites, we report them as regression tests, not as accuracy estimates.

Held-out set 1 (gate v2). Fresh notes written independently of the gate rules, on tables the suites never used: 21 notes on 5 tables (526 numeric mentions) for Level 1, and 8 notes with 294 extractor-style claims on 8 tables for Level 2. At Level 1 the gate raised 22 false alarms on 353 correct numbers (6.2%): 14 from gaps in the operation library (for example average order value for the whole table, or the rest of the table after the top k), 5 from rule gaps (for example "gave up" read as a rise) and 3 from entity names written with different spacing. It caught 79 of 111 wrong numbers (71%). Of the 32 misses, 21 are real cells attached to the wrong row or period, which Level 1 cannot see by design, and 11 are rule gaps: 6 direction verbs missing from the lists ("plummeted", "rebounded"), 4 hedged bounds and 1 number written as "c.137%". Excluding the by-design misses, it caught 88%. At Level 2 it raised 8 false alarms on 202 correct claims (4.0%), 4 of them from a single paraphrased product name matched to the wrong row, and detected 75 of 77 wrong claims (97%), with the error type right for 71 of 75. Of 10 group figures, 6 were verified and 4 were falsely flagged because their subsets are not in the library. The scripts for this set were written in a temporary folder that was later deleted, so only its recorded results survive; set 2 is saved in the repository.

Held-out set 2 (gate v3). Fresh notes produced by generator scripts (no model calls) that an AI agent wrote without seeing the gate's rules, following the note-format prompt, on 15 tables not used by the stress suites or the pilot (whether held-out set 1 used any of them cannot be checked, because its scripts were deleted), frozen by checksum before gate v3 (also checksummed) scored them; files in `paper/heldout2/`. Level 1 (28 notes, 681 numeric mentions): 7 false alarms on 435 correct numbers

(1.6%, Wilson 95% interval 0.8% to 3.3%), from a level followed by "down from" being signed negative (3; for example "£8,735.49 TW, down from £12,102.54 LW"), plural or one-word family names (3) and the mean of the rest of the table, which the library lacks (2); one false alarm had two of these causes. It caught 90 of 146 wrong numbers: 87 of 98 value errors (89%; all 13 direction errors) but 3 of 48 binding errors, which Level 1 cannot see by design. Level 2 (18 notes, 417 claims written as a careful extractor would, so extractor errors are excluded): 6 false alarms on 270 correct claims (2.2%, all per-order averages read as revenue), 3 correct claims lost as Unverifiable (a missing alias, UAE), and 99 of 104 wrong claims detected (95%) with the error type right in all 99; 21 of 24 correct group figures and all 9 wrong ones were labelled correctly. AI auditor agents re-derived a stratified random sample of 70 items per level from the tables and agreed with the recorded answer in all 140; these audits were not blind, since the recorded answers were available to the auditors. Set 2 differs from set 1 in style (its notes follow the note-format rules, as real model output does) as well as in gate version, so the drop in false alarms from set 1 to set 2 is not a clean estimate of what v3's fixes achieved.

Adjudication of the pilot. An AI agent checked, item by item against the tables and with programmatic lookups of each cell (not a human audit), every item the gate flagged in the pilot (3 unsupported numbers and 13 wrong claims under the gate at that point) and seeded random samples of 60 supported numbers and 60 correct claims. The 3 unsupported numbers were real errors, all in one sentence: a total for six parasol and umbrella lines reported as £14,211.15 TW versus £14,461.70 LW, a 1.7% decline, where the six rows sum to £17,211.15 and £16,325.70, a 5.4% rise. All 13 wrong claims were false alarms with three causes: group totals the extractor labelled TOTAL (6), a value found inside a longer number, such as 33 inside £680.33 (4), and absolute changes read as levels, such as "a decline of £51.1k versus LW" (3). The final gate fixes all three; on the pilot it flags exactly the 3 real errors at Level 1 and the same 3 as wrong group figures at Level 2, and nothing else. Because these fixes were made after seeing the pilot's flags, this agreement is not independent evidence of accuracy. The samples found no missed errors (0 of 60 numbers and 0 of 60 claims; a 95 percent upper bound of about 4.9 percent on the miss rate). One policy choice matters at this scale: six group shares equal the sum of the table's rounded Share_Pct cells but miss the exact share at the cited precision (for example 37.2% against an exact 37.13%); we accept them as derivable from the table as shown, and counting them as errors would raise the pilot's unsupported count from 3 to 9 numbers (from 0.18 to 0.55 percent of checked numbers).

Human audit. The harness also samples 200 numbers and 200 claims, stratified by strategy and label, for an audit by people and reports per-label precision and recall. No human audit has been done yet. The checks in this paper, including the full study's blind review, were made by AI agents with programmatic access to the tables, so a human spot check of a random subset of their verdicts is still needed before the error rates are treated as final.

9 Full study

Pre-registration. The analysis plan (`paper/preregistration.md`) was committed to the repository before any full-study note was generated. It froze gate v3 by checksum, together with the prompts and tables (the analysis and review code were written afterwards); fixed the design, the outcomes, four directional hypotheses and the statistics; and named a primary set. The gate's rules were revised on the pilot's notes, and on the six pilot tables the full study's Opus 5 notes are the pilot notes themselves, returned from the cache, so the **primary set is the 54 tables the pilot did not use**. All 60 tables are a secondary analysis. Analyses the plan does not list are marked exploratory below and collected in the Limitations. Two limits on this design should be stated plainly. First, the plan's only timestamp is a local commit, made ten seconds before the first test note and two minutes before the full run began; it was first pushed to the public repository together with the results, about two hours later, so readers have to take the order on trust. Second, the primary set still contains 24 tables that were used while developing or testing the gate: 9 tables of the regression stress suites and the 15 tables of held-out set 2. The notes on them are new, but the tables are not, so Table 7 repeats the strategy contrasts on the other 30 tables. The 13 tables of held-out set 1 cannot be identified, because its scripts were deleted, so those 30 tables may still include some.

Design. Three writers (Claude Opus 5, Claude Sonnet 5 and Claude Haiku 4.5, through the Anthropic API), the four strategies, 60 tables (20 per size) and two repeats give 1,440 notes. Opus 5 and Sonnet 5 do not accept a temperature, so they sample at the API default; Haiku 4.5 runs at temperature 0, so its two repeats are nearly identical. Claude Haiku 4.5 extracts the claims. No S2 script failed twice (0 failures) and no claim extraction failed (0), so no note met a pre-registered exclusion. Opus 5 refused part of the time: 20 of its notes (16 in the primary set) come from a generation that ended with the stop reason "refusal", all on 7 small country tables (9 direct-writing notes, the 9 write-then-verify notes built on those drafts, and 2 compute-then-write notes). The runner scored these notes like any other (two of the write-then-verify notes were rebuilt in full by revision; the rest are truncated); 4 of them contain no number at all, which is why two Opus 5 cells have 106 notes rather than 108, and none of them contains a confirmed error. Sonnet 5 and Haiku 4.5 never refused. On all 60 tables, notes met the 6 to 8 bullet format in 96.0% of cases for Opus 5, 97.1% for Sonnet 5 and 99.2% for Haiku 4.5. Most S3 notes are the S1 note unchanged: 98, 104 and 92 of 108 per writer in the primary set, either because Level 1 flagged nothing in the draft (92, 85 and 92 notes) or because the writer returned a flagged draft word for word (6 of 16, 19 of 23 and 0 of 16 drafts sent back). Each S3 minus S1 contrast therefore rests on a few revised notes (10, 4 and 16, on 8, 4 and 8 tables). On all 60 tables the notes contain 46,866 numeric mentions, of which 44,087 were checked; the gate flagged 343, 317 of them in the primary set. The API bill for generation and claim extraction was USD 39.86, under the USD 60 cap, summed from the cost recorded with every new response in the cache (it includes a small preflight test). Per-note costs add up to $58.56 because calls repeated from the pilot, the S1 drafts that S3 starts from, and claim extractions shared by identical notes were served from the cache but are counted in every note that uses them.

Blind review. The gate's flags were checked by review rather than trusted. A mention counts as flagged when Level 1 marks it Unsupported or Level 2 marks its claim Wrong_entity, Wrong_metric or Wrong_value. Review items were built for every flagged mention, for a seeded random sample of 300 mentions the gate supported and for 200 mentions whose claims it marked Correct (both stratified by writer), and shuffled. An item shows only the note's lines, the marked number and the path to the source table: not the writer, the strategy or the gate's verdict. Two reviewers worked through each batch of 60 items independently, one from the first item and one from the last, following a written guide

(`paper/adjudication/REVIEW_GUIDE.md`) that fixes the rounding rule (a number must match at the precision it is written to), hedges, direction words and how derived values are recomputed. A third reviewer decided the items on which the first two disagreed. The reviewers were Claude agents (Claude Opus 5.5, from the same model family as the writers) that read every table cell with pandas and wrote down their computation; they are not people. Blinding was enforced by the agents' instructions, not by withholding files, and note style can reveal some strategies (the facts-template notes read alike). Of 843 items, the first two reviewers agreed on 99.5%, which measures the consistency of one model with itself rather than the accuracy of its verdicts. In the primary set, 4 flags were judged unclear; they are not counted as errors, and counting them would raise the confirmed rates below only to 0.23%, 0.06% and 0.49%. The guide, which was written with the review and not pre-registered, counts a number as wrong when it fails at the precision written; how strict that is matters for one class of numbers, discussed under the rounding policy below.

Outcomes. The main outcome is the *confirmed error rate*: flagged mentions the review judged wrong, over checked numbers. *Gate precision* is confirmed errors over flags, and the *miss rate* is the share of sampled unflagged mentions judged wrong. Intervals come from a cluster bootstrap that resamples tables (2,000 resamples, seed 42). Strategy effects are paired differences against S1 on the same tables, with a Holm correction across the three contrasts of each writer; writer effects are the same against Opus 5 within each strategy; the intervals are not adjusted for multiplicity, the p values are. Gate precision and the miss rate use exact (Clopper-Pearson) intervals. The bootstrap p value is twice the smaller share of resamples on either side of zero. With so few errors per cell this test is anti-conservative, and a cell with no error gets a bootstrap interval of zero width, so Table 4 gives the exact upper bound for such cells. Table 7 repeats every strategy contrast with two further tests. The first is conditional within tables: given a table's total errors in the two arms, equal rates make one arm's count binomial in proportion to its checked numbers (20,000 simulations). It treats errors within an arm as independent, so it is anti-conservative when they cluster in notes, as Haiku 4.5's near-duplicate repeats make them. The second is a sign test over the tables that have any error, which makes no such assumption. These additions were not in the plan.

Table 4. Full study, primary set (54 tables). Strategies: S1 Direct, S2 Compute-then-write, S3 Write-then-verify, S4 Facts-template. Gate flags: mentions the gate flagged, as a percentage of checked numbers. Confirmed: flags the blind review judged wrong. Precision: confirmed errors over flags. Notes with error: notes containing at least one confirmed error. Brackets give 95 percent intervals from resampling tables. * No confirmed error: the bracket gives the exact (Clopper-Pearson) 95 percent upper bound, because the bootstrap interval collapses to zero; bootstrap intervals for cells with one or two errors are also too narrow.

Writer	Strategy	Checked	Gate flags (%)	Confirmed	Confirmed (%)	Precision	Notes with error
Opus 5	S1	4,870	0.64 [0.33, 0.99]	6	0.12 [0.02, 0.24]	6/31	6/106
	S2	2,677	1.57 [0.90, 2.29]	27	1.01 [0.45, 1.66]	27/42	13/108
	S3	4,943	0.38 [0.16, 0.65]	1	0.02 [0.00, 0.06]	1/19	1/106
	S4	2,725	0.11 [0.00, 0.34]	0	0.00 [0.00, 0.14]*	0/3	0/108
Sonnet 5	S1	3,888	1.00 [0.56, 1.54]	4	0.10 [0.03, 0.21]	4/39	4/108
	S2	2,329	0.56 [0.25, 0.91]	2	0.09 [0.00, 0.22]	2/13	2/108
	S3	3,887	0.90 [0.45, 1.44]	0	0.00 [0.00, 0.09]*	0/35	0/108
	S4	2,685	0.19 [0.04, 0.37]	1	0.04 [0.00, 0.11]	1/5	1/108
Haiku 4.5	S1	3,869	1.40 [0.72, 2.16]	31	0.80 [0.30, 1.42]	31/54	18/108
	S2	1,770	2.03 [1.05, 3.11]	17	0.96 [0.38, 1.69]	17/36	15/108
	S3	3,845	0.73 [0.36, 1.13]	6	0.16 [0.00, 0.35]	6/28	6/108
	S4	2,412	0.50 [0.09, 1.01]	2	0.08 [0.00, 0.25]	2/12	2/108

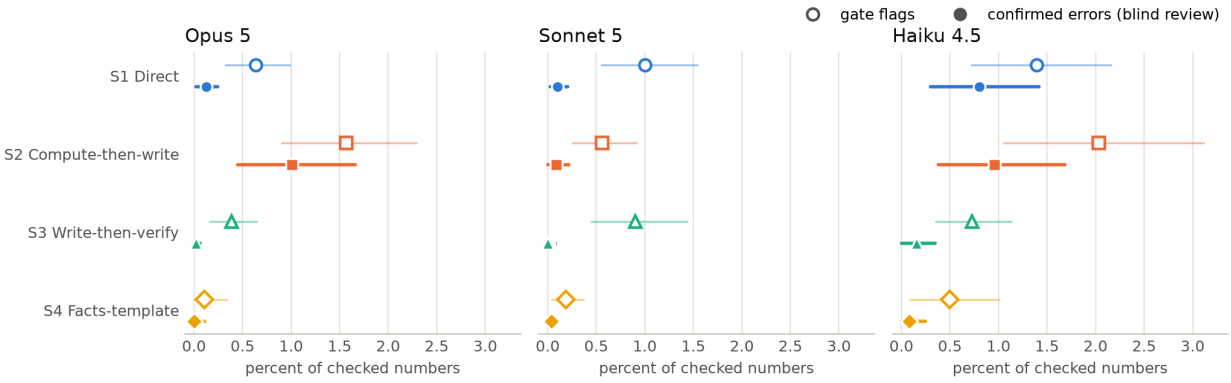


Figure 2. Gate flag rate (hollow markers) and confirmed error rate (filled markers) for each writer and strategy, primary set, with 95 percent intervals from resampling tables. The gap between the two markers is the gate's false alarms (and a few flags judged unclear). A dotted whisker marks a cell with no confirmed error and shows its exact upper bound.

Error rates. Table 4 and Figure 2 give the results. Confirmed errors are rare for all three writers, and because a number can only be confirmed if the gate flagged it, these rates are lower bounds (see the gate's recall below): 0.22% of checked numbers for Opus 5 (34 of 15,215), 0.05% for Sonnet 5 (7 of 12,789) and 0.47% for Haiku 4.5 (56 of 11,896). At the note level, which is what a reader experiences, errors concentrate in three cells: 18 of 108 Haiku 4.5 notes under direct writing and 15 of 108 under compute-then-write contained a wrong number, as did 13 of 108 Opus 5 notes under compute-then-write. Sonnet 5 wrote at most 4 of 108 notes with an error under any strategy. On all 60 tables (the secondary analysis) the rates are almost the same: 0.22%, 0.05% and 0.50%.

Table 5. Strategy contrasts in the confirmed error rate, primary set: each strategy minus direct writing (S1) on the same tables, in percentage points, with 95 percent intervals from resampling tables and *p* values corrected by Holm's method within each writer. Opus 5's contrasts use 53 tables, because both of its direct-writing notes on one table contain no number.

Writer	Contrast	Difference (points)	*p* (Holm)
Opus 5	S2 minus S1	0.90 [0.34, 1.59]	<0.001
	S3 minus S1	−0.10 [−0.21, −0.02]	0.018
	S4 minus S1	−0.12 [−0.24, −0.04]	0.018
Sonnet 5	S2 minus S1	−0.02 [−0.17, 0.15]	0.795
	S3 minus S1	−0.10 [−0.21, −0.03]	0.075
	S4 minus S1	−0.07 [−0.18, 0.06]	0.542
Haiku 4.5	S2 minus S1	0.16 [−0.73, 1.08]	0.725
	S3 minus S1	−0.65 [−1.20, −0.20]	0.003
	S4 minus S1	−0.72 [−1.35, −0.17]	0.012

Strategies. Table 5 compares each strategy with direct writing using the pre-registered test, and Table 7 repeats each comparison with other tests and other definitions of an error. Compute-then-write did not help any writer and raised Opus 5's error rate by +0.90 points (p <0.001), making it Opus 5's worst strategy. The increase is significant under the conditional test for every definition of an error and without the development tables, but it comes from 11 tables that got worse against 4 that got better (sign test p = 0.118), and without rounding slips the pre-registered bootstrap no longer separates it (+0.27 points, Holm p = 0.108). Write-then-verify and the facts template had fewer confirmed errors than direct writing for Opus 5 and Haiku 4.5: the differences are −0.10 and −0.12 points for Opus 5 (Holm p = 0.018 and 0.018) and −0.65 and −0.72 points for Haiku 4.5 (p = 0.003 and 0.012). For Opus 5 these reductions are six rounding slips under direct writing against one and none. They are not significant under the conditional test (Holm p = 0.141 and 0.141) or the sign test (0.125 and 0.062), they disappear without rounding slips, and under the pilot's rounding rule they shrink to one error against none. For Haiku 4.5 the reductions hold under the conditional test (Holm p <0.001 and <0.001) and the sign test (p = 0.016 and 0.021), and under the conditional test also without rounding slips, under the pilot's rule and without the development tables. They are less secure than that list suggests: without rounding slips the pre-registered bootstrap no longer separates them (Holm p = 0.132 and 0.132), and the conditional test is anti-conservative for Haiku 4.5, whose repeats are near-copies. For Sonnet 5, whose direct notes had only 4 confirmed errors, no contrast is significant, although write-then-verify had none. Two caveats apply. For write-then-verify, most notes are copies of their S1 drafts, and S3 was revised against the same Level 1 rules that decide which of its numbers are reviewed. For every strategy, a per-number rate can favour notes that cite simpler numbers: compute-then-write and the facts template cite roughly half to two thirds as many checked numbers as direct writing, with fewer derived totals and shares, and the study has no measure of how informative a note is.

Hypotheses. **H1** (the facts template has the lowest confirmed error rate for every writer) is not supported as stated, because it fails for Sonnet 5, where write-then-verify had no error and the facts template one. The facts template does have the lowest rate for Opus 5 (0 errors, 0.00%) and Haiku 4.5 (2 errors, 0.08%), but the plan contains no test of the facts template against write-then-verify, and their rates do not differ clearly for any writer. **H2** (gate precision below 1, most of all for the strongest writer) is supported in its first part, which any checker with false alarms would satisfy: precision was 35.8% for Opus 5 (95 percent interval 26.2% to 46.3%), 7.6% for Sonnet 5 (3.1% to 15.1%) and 43.1% for Haiku 4.5 (34.4% to

52.0%). The second part depends on a term the plan did not define. If the strongest writer is the one with the fewest confirmed errors, Sonnet 5, it is supported; if it is the largest model, Opus 5, it is not, because Opus 5's precision was far above Sonnet 5's and not clearly below Haiku 4.5's. **H3** (under direct writing, more errors on large tables than on small ones) is not supported: large minus small was −0.08 points for Opus 5 (interval −0.40 to +0.26), +0.16 for Sonnet 5 (+0.00 to +0.40) and +0.57 for Haiku 4.5 (−0.60 to +2.10), with no p below 0.236. With 18 primary tables per size and few errors this test has little power, so the result is inconclusive rather than evidence of no effect; table size is also confounded with entity type, since all country tables are small. **H4** (under direct writing, Haiku 4.5 errs more than Opus 5) is supported: the difference is +0.69 points (+0.18 to +1.32, Holm $p = 0.012$), although Haiku 4.5 also ran at temperature 0 and Opus 5 at the default, so the comparison mixes model and decoding. Table 6 gives all eight pre-registered writer contrasts; the only other significant one is Sonnet 5 against Opus 5 under compute-then-write (−0.92 points, $p < 0.001$).

Table 6. All pre-registered writer contrasts in the confirmed error rate, primary set: each writer minus Opus 5 within a strategy, in percentage points, with 95 percent intervals from resampling tables and Holm-corrected p values.

Strategy	Contrast	Difference (points)	p (Holm)
S1	Sonnet 5 minus Opus 5	−0.02 [−0.12, 0.09]	0.729
	Haiku 4.5 minus Opus 5	0.69 [0.18, 1.32]	0.012
S2	Sonnet 5 minus Opus 5	−0.92 [−1.52, −0.40]	<0.001
	Haiku 4.5 minus Opus 5	−0.05 [−0.87, 0.78]	0.906
S3	Sonnet 5 minus Opus 5	−0.02 [−0.06, 0.00]	0.709
	Haiku 4.5 minus Opus 5	0.14 [−0.02, 0.35]	0.234
S4	Sonnet 5 minus Opus 5	0.04 [0.00, 0.11]	1.000
	Haiku 4.5 minus Opus 5	0.08 [0.00, 0.25]	1.000

Table 7. Sensitivity of the strategy contrasts, primary set (exploratory). Pre-registered: difference in points with its bootstrap Holm p (Table 5). Conditional p: conditional test within tables (Section 9, Outcomes), Holm-corrected within writer. Tables up/down: tables where the strategy had a higher or lower error rate than S1, with an exact sign-test p. The last three columns give the difference and conditional Holm p when rounding slips are not counted, when sums of rounded shares are accepted as the pilot did, and on the 30 primary tables not used by the stress suites or held-out set 2.

Writer	Versus S1	Pre-registered	Conditional p	Tables up/down (sign p)	Without rounding slips	Pilot rounding rule	Without development tables
Opus 5	S2	0.90 (<0.001)	<0.001	11/4 (0.118)	0.27 (0.003)	0.97 (<0.001)	1.06 (<0.001)
	S3	−0.10 (0.018)	0.141	0/4 (0.125)	0.00 (1.000)	−0.02 (1.000)	−0.03 (1.000)
	S4	−0.12 (0.018)	0.141	0/5 (0.062)	0.00 (1.000)	−0.02 (1.000)	−0.07 (1.000)
Sonnet 5	S2	−0.02 (0.795)	1.000	2/4 (0.688)	−0.02 (1.000)	−0.02 (1.000)	−0.06 (1.000)
	S3	−0.10 (0.075)	0.390	0/4 (0.125)	−0.10 (0.384)	−0.10 (0.390)	−0.14 (0.762)
	S4	−0.07 (0.542)	0.844	1/4 (0.375)	−0.07 (0.845)	−0.07 (0.844)	−0.07 (1.000)
Haiku 4.5	S2	0.16 (0.725)	0.653	8/8 (1.000)	0.03 (1.000)	0.21 (0.446)	0.57 (0.115)
	S3	−0.65 (0.003)	<0.001	0/7 (0.016)	−0.44 (0.006)	−0.59 (<0.001)	−0.45 (0.006)
	S4	−0.72 (0.012)	<0.001	1/9 (0.021)	−0.51 (0.006)	−0.67 (0.001)	−0.45 (0.038)

Table 8. Kinds of confirmed error, primary set (exploratory; strategies as in Table 4). Rounding: a rounding or precision slip, off only in the last written digit, for example a share computed by adding already-rounded shares. Direction: the right magnitude called a decline when it was a rise, or the reverse. Value: a computed total, share or change that is off by more than rounding. Scope: a real value from the wrong row, column or period, or a claim stretched over rows it does not fit.

Writer	Strategy	Rounding	Direction	Value	Scope	Total
Opus 5	S1	6	0	0	0	6
	S2	20	7	0	0	27
	S3	1	0	0	0	1
	S4	0	0	0	0	0
Sonnet 5	S1	0	0	4	0	4
	S2	0	2	0	0	2
	S3	0	0	0	0	0
	S4	0	0	0	1	1
Haiku 4.5	S1	8	0	20	3	31
	S2	6	0	4	7	17
	S3	0	0	2	4	6
	S4	0	0	0	2	2

What the errors are (exploratory). After the review, the AI assistant that ran the analysis sorted each confirmed error into one of four kinds, using the reviewers' written computations (Table 8; the labels are in `paper/adjudication/error_types_full.csv`). Of 97 confirmed errors in the primary set, 41 were rounding or precision slips, 9 had the wrong direction, 30 were wrong computed values (mostly Haiku 4.5's totals and top-ten shares under direct writing) and 17 used a real value from the wrong row, column or period, for example last year's revenue given as this week's. Opus 5's compute-then-write errors are 20 rounding slips and 7 wrong directions, and each one traces to the facts script the model wrote. Nineteen of the rounding slips are money changes that the script rounded to one decimal and the writer printed with a trailing zero, as if exact to the penny (£1,213.70 where the exact change is £1,213.69); the twentieth is a share the script computed by adding rounded shares. The wrong directions come from five notes whose scripts labelled the smallest of a set of rises the "largest drop", which the writer repeated. Computing before writing moves the risk into the code, where a careless rounding or label is copied faithfully into the note. Rounding slips are real errors at the precision written, but they are small. Setting them aside, the rates are 0.05% for Opus 5 (7 errors, all wrong directions under compute-then-write), 0.05% for Sonnet 5 and 0.35% for Haiku 4.5. Opus 5 then has no error at all under direct writing, write-then-verify or the facts template, so its reductions disappear, while Haiku 4.5's reductions and Opus 5's compute-then-write increase remain under the conditional test (Table 7).

Rounding policy (exploratory). Which shares count as rounding errors is a policy choice that this study did not apply consistently. The pilot adjudication and gate v3 accept a named group's share when it equals the sum of the table's rounded Share_Pct cells; the full-study guide counts any share that fails at the written precision. The same arithmetic is therefore confirmed as an error when a note says "the top five" and accepted, without review, when the note names the rows. Of the confirmed errors in the primary set, 9 are such sums, including five of Opus 5's six direct-writing errors and its only write-then-verify error. Under the pilot's rule Opus 5 has 1, 0 and 0 errors under direct writing, write-then-verify and the facts template; under the strict rule applied to every number, some gate-accepted sums would be added that were never reviewed. The choice decides whether the pre-registered test shows any reduction for

Opus 5 under write-then-verify and the facts template; the compute-then-write increase holds under either rule. A single, stated policy should be fixed before the next study.

Table 9. The gate's own rates (pre-registered outcomes 1 and 2), primary set: unsupported numbers (Level 1), binding errors (Wrong_entity and Wrong_metric claims) and all claim errors (binding plus Wrong_value), as percentages, with 95 percent intervals from resampling tables (* no event: the exact upper bound instead); strategies as in Table 4. These are raw gate rates, before review.

Writer	Strategy	Numbers checked	Unsupported (%)	Claims verifiable	Binding errors (%)	Claim errors (%)
Opus 5	S1	4,870	0.45 [0.22, 0.73]	4,870	0.08 [0.02, 0.17]	0.51 [0.21, 0.87]
	S2	2,677	0.86 [0.33, 1.44]	2,676	0.37 [0.11, 0.68]	1.53 [0.89, 2.26]
	S3	4,943	0.20 [0.06, 0.40]	4,949	0.08 [0.02, 0.17]	0.24 [0.06, 0.45]
	S4	2,725	0.00 [0.00, 0.14]*	2,724	0.11 [0.00, 0.34]	0.11 [0.00, 0.34]
Sonnet 5	S1	3,888	0.82 [0.49, 1.26]	3,891	0.18 [0.00, 0.52]	0.28 [0.05, 0.63]
	S2	2,329	0.17 [0.04, 0.35]	2,330	0.21 [0.00, 0.47]	0.39 [0.13, 0.73]
	S3	3,887	0.72 [0.37, 1.11]	3,891	0.18 [0.00, 0.51]	0.18 [0.00, 0.52]
	S4	2,685	0.07 [0.00, 0.19]	2,683	0.11 [0.00, 0.26]	0.11 [0.00, 0.26]
Haiku 4.5	S1	3,869	0.57 [0.21, 1.04]	3,937	0.66 [0.26, 1.13]	1.42 [0.71, 2.22]
	S2	1,770	0.90 [0.34, 1.64]	1,784	0.67 [0.00, 1.56]	1.91 [0.92, 3.01]
	S3	3,845	0.00 [0.00, 0.10]*	3,907	0.67 [0.26, 1.16]	0.77 [0.35, 1.24]
	S4	2,412	0.25 [0.00, 0.51]	2,480	0.24 [0.00, 0.65]	0.24 [0.00, 0.64]

How good is the gate on real notes? Table 9 gives the gate's raw rates. Of its flags in the primary set, 97 of 317 were real errors (30.6%, 95 percent interval 25.6% to 36.0%). None of the 451 sampled unflagged mentions in the primary set was wrong (95 percent upper bound 0.8%; the other 49 of the 500 sampled mentions are on pilot tables). That bound is not small: applied to the 39,583 unflagged numbers it allows up to about 322 missed errors against 97 confirmed, so the gate's recall could be as low as 23%, and the sample, stratified by writer only, cannot show whether recall differs between strategies. The per-writer upper bounds on the true error rate, adding the worst plausible misses to the confirmed errors, are 2.61% for Opus 5, 2.44% for Sonnet 5 and 2.92% for Haiku 4.5. One known class of misses is the rounded-share sums the gate accepts. The review also left out Level 2 flags on claims that could not be matched to a number in the text, and one real Sonnet 5 error was matched to the wrong occurrence of its number.

Precision depends on how rare errors are and on how often the gate raises false alarms on correct text. Using the false alarm rate measured on held-out set 2, the pilot predicted that only about 9% of flags would be real at its base rate (Section 10). On real notes the gate raised fewer false alarms than on the held-out notes: it flagged 0.40% of Opus 5's numbers that were not confirmed errors (counting Level 1 and Level 2 flags, whereas the held-out rate counts Level 1 only), so at a similar base rate Opus 5's precision was 35.8%. At Level 1 alone, which is what the held-out rate measures, the gate flagged 0.17%, 0.49% and 0.12% of the three writers' numbers that were not confirmed errors, far below the held-out 1.6%. This is consistent with the first of the two readings the pilot left open (Section 10): the held-out notes were built to exercise many phrasings, and real notes use fewer of those that trip the gate. Counting both levels, the false alarm rate was higher for the other writers (0.66% for Sonnet 5 and 0.61% for Haiku 4.5). Haiku 4.5's excess is at Level 2, where 58 of its 72 false alarms arise and where it also extracts its own claims. With Sonnet 5's low error rate, only 7.6% of its flags were real. Its false alarms have a single

main cause: of Sonnet 5's 62 Level 1 false alarms, 50 came from one phrasing, a level followed by a change, as in "TW revenue of £5735.62, down from LW £9020.33": gate v3 attaches "down" to the level before it, signs it negative and fails to match a correct cell. This is not a new gap: held-out set 2 had already traced 3 of its 7 Level 1 false alarms to it, but only Sonnet 5 used the phrasing often enough for it to dominate the flags. The same phrasing caused 6 of Haiku 4.5's 14 Level 1 false alarms and none of Opus 5's 26, and none of the flags it caused was a real error. Because of it, raw gate rates cannot tell Sonnet 5 from Opus 5. The gate flagged 0.72% of Sonnet 5's checked numbers and 0.62% of Opus 5's, a difference of +0.09 points (95 percent interval −0.28 to +0.49, $p = 0.664$; exploratory). Confirmed errors are lower for Sonnet 5 (0.05% against 0.22%, a difference of −0.17 points, −0.27 to −0.08), but that gap comes from Opus 5's compute-then-write errors: without that strategy it is −0.01 points ($p = 0.727$), and without rounding slips the two writers have the same number of errors. Within Sonnet 5, the gate flagged write-then-verify almost as often as direct writing (0.90% against 1.00%), although write-then-verify had no confirmed error. The gate stays frozen as pre-registered (Section 10).

Reliability across repeats. Whether a note contains a confirmed error agreed between the two repeats of the same table and strategy with Cohen's $\kappa = 0.27$ for Opus 5 (93.5% agreement), −0.01 for Sonnet 5 (96.8%; errors are too rare for κ to say much) and 0.97 for Haiku 4.5 (99.5%). Haiku 4.5's high value reflects temperature 0, under which its two repeats are nearly the same note, not stability. These values have no intervals and include S3 notes that copy their S1 drafts. For Opus 5, which samples, an error seldom recurred: of the 17 pairs of repeats in which a note had a confirmed error, both notes had one in only 3. Which note contains an error varies from one sample to the next, so single notes say little and rates should be pooled over many. Because the bootstrap resamples tables, the near-duplicate Haiku 4.5 repeats do not narrow its bootstrap intervals, although the exact intervals for gate precision treat its duplicated flags as independent; they also treat flags on S3 notes copied from their S1 drafts as independent, so every precision interval is somewhat too narrow.

10 Discussion and limitations

What the results say. For the three Claude models, this prompt and these tables, confirmed errors were rare: 0.22%, 0.05% and 0.47% of checked numbers across strategies. These are lower bounds, and they hide the cells that matter: under direct writing Haiku 4.5 reached 0.80%, and under compute-then-write Haiku 4.5 and Opus 5 reached 0.96% and 1.01%. Allowing for the errors the gate may miss, the true rates could be as high as 2.44% to 2.92%. The errors are not spread evenly. Haiku 4.5 miscomputes totals and shares when it writes directly from the table (20 of its 31 direct-writing errors) and, under every strategy, sometimes uses real values from the wrong row or column. Write-then-verify and the facts template reduced the miscomputed totals and shares, though not the wrong-row errors (Table 8), and that reduction holds under the conditional and sign tests; Opus 5's smaller reductions do not. Asking the model to compute its facts in code first did not help, and for Opus 5 it introduced errors of its own: a script that rounds or labels carelessly passes its mistakes straight into the note. Of the strategies tested, narrating facts computed in advance by deterministic code (the facts template) gave the lowest or near-lowest confirmed rate for every writer; checking the note against the table after writing (write-then-verify) clearly helped only Haiku 4.5, and part of its measured benefit is built in, because S3 is revised against the gate that selects its numbers for review. Code written by the model on the fly (compute-then-write) helped no writer and hurt Opus 5.

What the gate's own error rate implies. The pilot's true error rate (3 wrong numbers in 1,625, 0.18%) is far below the gate's false alarm rate on fresh held-out text: 6.2% of correct numbers and 4.0% of correct claims for gate v2 on set 1, and 1.6% and 2.2% for gate v3 on set 2. Bayes' rule makes the

consequence concrete. With set 2's false alarm rate (1.6%) and value-error recall (89%), a Level 1 flag at the pilot's base rate would be a real error only about 9% of the time, roughly 10 false flags for every real one. The pilot's three flags may all have been real because the gate's rules had been revised on those very notes. Two readings are consistent with our data: real model notes may use fewer of the phrasings that trip the gate than our held-out notes, which were built to exercise many phrasings, do; or the pilot's zero false alarms reflect rules revised on the pilot itself. Either way, raw gate rates cannot be compared across models or strategies at this scale without an estimate of the gate's precision on the same notes. The full study bears this out. The first reading fits all three writers at Level 1, and Opus 5's notes tripped the gate far less often than the held-out notes did, so the gate's precision on its notes (35.8%) was much higher than the pilot predicted. But one phrasing habit, a known gate gap that held-out set 2 had shown only in a few notes, made most of Sonnet 5's flags false, and raw gate rates could not separate the two writers. A checker's precision is therefore a property of the checker and the writer together; it has to be measured on the notes being scored, and it cannot be carried over from a validation set or from another model. We treat the gate as a triage tool whose flags are confirmed by review, and we report confirmed errors, not gate rates, as the main outcome. Two standard tools, and related methods, would go further. A prevalence correction for an imperfect test [26], or prediction-powered inference [3] and the related methods of Deriu et al. [7] and Gligorić et al. [12], applied with a fully reviewed random subset of notes, would turn the confirmed lower bounds into estimates of the true error rate with valid intervals; this needs a larger gold-standard sample, ideally labelled by people, than this study has.

Limitations.

- **The reviewers are not people.** Every check in this paper, including the full study's blind review, was made by Claude agents that recomputed each number from the table with code. Their 99.5% agreement is one model agreeing with itself; they come from the same model family as the writers and could share blind spots; blinding was enforced only by their instructions; and their guide counts a number as wrong when it fails at the precision written, a strict rule a human reader might relax for a penny. A human audit of a random subset of verdicts is still needed.
- **Inconsistent rounding policy.** Sums of rounded shares are accepted by the pilot adjudication and by the gate but counted as errors by the full-study review; the choice decides whether the pre-registered test shows any reduction for Opus 5 under write-then-verify and the facts template (Section 9).
- **Refusals and copied notes.** Opus 5 refused on 20 notes: two were rebuilt in full by revision and the rest were scored as truncated notes, 4 of them with no number. None contains a confirmed error, but truncation removes numbers from Opus 5's small-country notes, and copied write-then-verify notes shrink the effective sample for the comparisons they enter.
- **Development tables in the primary set.** 24 primary tables were used, with other notes, while developing or testing the gate. On the remaining 30 tables the strategy contrasts keep their direction and the same ones stay significant under the conditional test (Table 7), but held-out set 1's tables could not be excluded.
- **Errors the gate misses.** The review checked every flagged number but only a sample of 451 unflagged mentions. No sampled mention was wrong, but the per-writer samples are small, so the upper bounds on each writer's true error rate (2.61%, 2.44% and 2.92%) are far above the confirmed rates. Errors of a kind the gate cannot see, such as numbers written as words, are not counted at all.
- **One provider and three writers.** All writers, the claim extractor and the reviewers are Claude models, chosen for budget, and Haiku 4.5 extracts the claims of its own notes; other model families may err differently and trip the gate with other phrasings. Opus 5 and Sonnet 5 sample at the API

default temperature while Haiku 4.5 runs at temperature 0, so their repeats are not comparable samples, and Haiku 4.5's error rate rests on what is effectively one note per table and strategy.

- **Exploratory analyses.** The kinds of error and the analysis without them, the rounding-policy comparison, the conditional and sign tests, the exact bounds for empty cells, the analysis without development tables, counting unclear verdicts as errors, the pooled writer comparison, the phrasing behind Sonnet 5's false alarms, the recall and true-error-rate bounds, the exact intervals for gate precision and the count of S3 copies were not in the pre-registered plan. The error kinds were assigned after the review, by the AI assistant that ran the analysis, with the writer of each note known and without a second coder.
- **Pilot size.** 48 notes from one model on six tables; the intervals are wide and no strategy difference is significant. The final gate's rules were revised after seeing the pilot's flags, so its agreement with the adjudication on the pilot is not independent evidence; the held-out tests and the full study are.
- **One data domain.** A single UK online retailer, weekly grain, GBP, and country and product entities.
- **Known gaps in gate v3.** Held-out set 2 found: per-order averages cited as money are read as revenue at Level 2; a level followed by "down from" or a similar change phrase ("£8,735.49 TW, down from £12,102.54 LW") is signed negative, the gap behind most of Sonnet 5's false alarms; plural or one-word family names ("cushions", "MUG") are not recognised as families; the mean of the rest of the table is not in the library; bounds are not allowed on group figures; and some aliases (UAE) are missing. The gate stays frozen as pre-registered; fixing these gaps requires a new held-out set to measure the result honestly.
- **Gate coverage.** Numbers written as words ("doubled") are not extracted; derived values outside the operation library are Unsupported; paraphrased product names may bind to the wrong row (found in held-out set 1); superlatives ("the largest swing") are not checked; the 0.5 percent tolerance for k and M values accepts a wrong last digit at large scales; truncated values without k or M count as unsupported.
- **Extractor dependence.** Level 2 inherits the extractor's errors, and a value assigned to the wrong entity in a list can still produce a false Wrong_entity.
- **S3 and its scorer.** Write-then-verify is revised against the Level 1 gate that also scores it, so its raw Level 1 rate is biased low (Haiku 4.5's is zero in Table 9, although the review confirmed 6 errors in its notes, all caught at Level 2). The confirmed error rate is the fairer comparison, but it too only reviews what the gate flags.

11 Reproducibility

The code, data, prompts, gate rules, review files and an interactive dashboard of every run are public. The dashboard is at `https://harshitha-venkatesh-2204.github.io/NumGate/` and the repository at `https://github.com/harshitha-venkatesh-2204/NumGate`. The harness is plain Python with pandas. Tables are rebuilt from the UCI download with a fixed seed and every derived column is checked on build. Every model response is cached on disk, keyed by model, prompts, temperature and repeat, so rerunning any experiment is free; a budget cap stops a run and a rerun resumes it. `run_all` builds the tables, runs a smoke experiment with a mock model (no API key), analyses it and opens the dashboard. The pilot is reproduced by `python src/run_experiment.py --models claude-opus-5 --tables-per-size 2 --run-id smoke_opus`, the v1 comparison by `paper/rescore_v1.py`, and the pilot and validation numbers by `paper/make_numbers.py`. The full study is reproduced by `MAX_BUDGET_USD=60 python src/run_experiment.py --models claude-opus-5 claude-sonnet-5 claude-haiku-4-5 --run-id full`; its responses are in the released cache, so the rerun costs nothing, although the runner still asks for an API key before it starts. The analysis needs only `python paper/full_study.py`, which reads the released run and verdicts. The pre-registered plan is `paper/preregistration.md`. The review items are built by `paper/adjudication/build_items.py`, reviewed by the workflow saved in `paper/adjudication/review_workflow.js` under `REVIEW_GUIDE.md` (run with Claude Opus 5.5 agents; the model is not recorded in the released files), and collected by `collect_verdicts.py`; the items, the unblinding key, both reviewers' verdicts with their computations, and the final verdicts are all released. `paper/full_study.py` computes every full-study number and table in this paper from the run and the verdicts.

Use of AI

The research questions, the study design and the logic of the checker, the generation strategies and the analysis are the author's. The author used AI extensively to implement and carry out the work, in the style often called vibe coding. An AI coding assistant (Claude Code, running Anthropic's Claude models) wrote the code for the harness, the gate, the analysis and the dashboard, generated the test notes of the regression suites and held-out sets, ran the experiments, and drafted and revised the text of this paper under the author's direction. AI agents also carried out the checking work: the sorting of the first gate's false alarms into causes, the pilot adjudication, the audit of held-out set 2, the blind review of the full study, the sorting of its confirmed errors into kinds, and two independent critiques of a draft of this paper. The writers and the claim extractor studied here are Claude models too. The author takes responsibility for the content. AI tools are not authors of this paper.

References

[1] Peter Røysland Aarnes and Vinay Setty. Towards Robust Numerical Claim Verification. *arXiv preprint arXiv:2610.00689*, 2026. Accepted to Findings of AACL-IJCNLP 2026. `https://doi.org/10.48550/arXiv.2610.00689`

[2] Rami Aly and Andreas Vlachos. TabVer: Tabular Fact Verification with Natural Logic. *Transactions of the Association for Computational Linguistics*, 12:1648–1671, 2024. `https://doi.org/10.1162/tacl_a_00722`

[3] Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. Prediction-powered inference. *Science*, 382(6671):669–674, 2023. `https://doi.org/10.1126/science.adi6000`

[4] Daqing Chen. Online Retail II. *UCI Machine Learning Repository*, 2012. `https://archive.ics.uci.edu/dataset/502/online+retail+ii` `https://doi.org/10.24432/C5CG6D`

[5] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. TabFact: A Large-scale Dataset for Table-based Fact Verification. *Eighth International Conference on Learning*

Representations (ICLR 2020), 2020.

[6] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, 2023.

[7] Jan Deriu, Pius von Däniken, Don Tuggener, and Mark Cieliebak. Correction of Errors in Preference Ratings from Automated Metrics for Text Generation. *Findings of the Association for Computational Linguistics: ACL 2023*, pages 6456–6474, 2023. https://doi.org/10.18653/v1/2023.findings-acl.404

[8] Bhuwan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. Handling Divergent Reference Texts when Evaluating Table-to-Text Generation. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4884–4895, 2019. https://doi.org/10.18653/v1/P19-1483

[9] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-Verification Reduces Hallucination in Large Language Models. *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3563–3578, 2024. https://doi.org/10.18653/v1/2024.findings-acl.212

[10] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993. https://doi.org/10.1007/978-1-4899-4541-9

[11] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. *Proceedings of the 40th International Conference on Machine Learning*, 202:10764–10799, 2023.

[12] Kristina Gligorić, Tijana Zrnic, Cinoo Lee, Emmanuel Candès, and Dan Jurafsky. Can Unconfident LLM Annotations Be Used for Confident Conclusions? *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3514–3533, 2025. https://doi.org/10.18653/v1/2025.naacl-long.179

[13] Ameya Godbole and Robin Jia. Verify with Caution: The Pitfalls of Relying on Imperfect Factuality Metrics. *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22889–22912, 2025. https://doi.org/10.18653/v1/2025.findings-acl.1175

[14] Tomas Goldsack, Yang Wang, Chenghua Lin, and Chung-Chi Chen. From Facts to Insights: A Study on the Generation and Evaluation of Analytical Reports for Deciphering Earnings Calls. *Proceedings of the 31st International Conference on Computational Linguistics*, pages 10576–10593, 2025.

[15] Or Honovich, Roee Aharoni, Jonathan Herzig, Hagai Taitelbaum, Doron Kukliansy, Vered Cohen, Thomas Scialom, Idan Szpektor, Avinatan Hassidim, and Yossi Matias. TRUE: Re-evaluating Factual Consistency Evaluation. *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3905–3920, 2022. https://doi.org/10.18653/v1/2022.naacl-main.287

[16] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12):1–38, 2023. https://doi.org/10.1145/3571730

[17] Zdeněk Kasner and Ondřej Dušek. Beyond Traditional Benchmarks: Analyzing Behaviors of Open LLMs on Data-to-Text Generation. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12045–12072, 2024. https://doi.org/10.18653/v1/2024.acl-long.651

[18] Michael Krumdick, Charles Lovering, Varshini Reddy, Seth Ebner, and Chris Tanner. No Free Labels: Limitations of LLM-as-a-Judge Without Human Grounding. *Third Conference on Language Modeling*, 2026.

[19] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. https://doi.org/10.1162/tacl_a_00453

[20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. https://doi.org/10.52202/075280-2019

[21] Potsawee Manakul, Adian Liusie, and Mark Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, 2023. https://doi.org/10.18653/v1/2023.emnlp-main.557

[22] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, 2023. https://doi.org/10.18653/v1/2023.emnlp-main.741

[23] Artidoro Pagnoni, Vidhisha Balachandran, and Yulia Tsvetkov. Understanding Factuality in Abstractive Summarization with FRANK: A Benchmark for Factuality Metrics. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4812–4829, 2021. https://doi.org/10.18653/v1/2021.naacl-main.383

[24] Ankur Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. ToTTo: A Controlled Table-To-Text Generation Dataset. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1173–1186, 2020. https://doi.org/10.18653/v1/2020.emnlp-main.89

[25] Ratish Puduppully, Li Dong, and Mirella Lapata. Data-to-Text Generation with Content Selection and Planning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(01):6908–6915, 2019. https://doi.org/10.1609/aaai.v33i01.33016908

[26] Walter J. Rogan and Beth Gladen. Estimating Prevalence from the Results of a Screening Test. *American Journal of Epidemiology*, 107(1):71–76, 1978. https://doi.org/10.1093/oxfordjournals.aje.a112510

[27] Aivin V. Solatorio. Proof-Carrying Numbers (PCN): A Protocol for Trustworthy Numeric Answers from LLMs via Claim Verification. *arXiv preprint arXiv:2509.06902*, 2025. https://doi.org/10.48550/arXiv.2509.06902

[28] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryscinski, Justin Rousseau, and Greg Durrett. Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11626–11644, 2023. https://doi.org/10.18653/v1/2023.acl-long.650

[29] Craig Thomson and Ehud Reiter. A Gold Standard Methodology for Evaluating Accuracy in Data-To-Text Systems. *Proceedings of the 13th International Conference on Natural Language Generation*, pages 158–168, 2020. https://doi.org/10.18653/v1/2020.inlg-1.22

[30] Venktesh V, Abhijit Anand, Avishek Anand, and Vinay Setty. QuanTemp: A real-world open-domain benchmark for fact-checking numerical claims. *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 650–660, 2024. https://doi.org/10.1145/3626772.3657874

[31] Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. Challenges in Data-to-Document Generation. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2253–2263, 2017. https://doi.org/10.18653/v1/D17-1239

[32] Zheng Zhao, Shay B. Cohen, and Bonnie Webber. Reducing Quantity Hallucinations in Abstractive Summarization. *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2237–2249, 2020. https://doi.org/10.18653/v1/2020.findings-emnlp.203

[33] Yilun Zhao, Yitao Long, Tintin Jiang, Chengye Wang, Weiyuan Chen, Hongjun Liu, Xiangru Tang, Yiming Zhang, Chen Zhao, and Arman Cohan. FinDVer: Explainable Claim Verification over Long and Hybrid-content Financial Documents. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14739–14752, 2024. https://doi.org/10.18653/v1/2024.emnlp-main.818

[34] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023. https://doi.org/10.52202/075280-2020